

Zaks Foods Sales Portal: Admin Manual (v2 - Deep Dive)

1. System Architecture & Overview

The Zaks Sales Portal is an enterprise-grade Field Sales Application built to bridge the gap between remote sales representatives and Oracle NetSuite.

- **Frontend Layer:** Built on React and hosted on Vercel, ensuring high availability, speed, and mobile responsiveness.
- **Database Layer (Supabase):** Utilizes PostgreSQL for robust data storage. Features Row-Level Security (RLS) to ensure reps only see what they are permitted to see.
- **Integration Layer (NetSuite Suitelets):** Custom SuiteScript (JavaScript) APIs hosted directly in NetSuite that parse incoming CSV files and automatically generate native NetSuite records (Sales Orders, Invoices, Customers).

2. Comprehensive User & Access Management

The Role System

The application relies on a strict role-based access control (RBAC) system defined in the `user_roles` table in Supabase.

- **Sales Rep (Default):** Has access only to customers assigned to their specific territory/ID. Cannot view system logs or globally edit data.
- **Administrator:** Can view all customers across all territories, access the Admin Dashboard, and modify backend settings.

Creating and Provisioning a New User

1. Log in to the **Supabase Dashboard** and navigate to **Authentication > Users**.
2. Click **Add User** -> **Create New User**.
3. Enter the user's corporate email and a secure temporary password.
4. Uncheck "Auto Confirm User" to send a welcome email, or check it to silently provision the account.
5. Click **Create User**.
6. **(Crucial Step for Admins):** To grant Admin rights, navigate to the **Table Editor** -> `user_roles` table. Insert a new row mapping the new user's `user_id` to the `role_id` that corresponds to "ADMIN".

Credential Distribution

Since the portal is a private B2B tool, users must be given the direct URL:

`https://zaks-lion-sales-portal.vercel.app/`

If a user is locked out or forgets their password, instruct them to use the native "Forgot Password" flow on the login screen, which securely triggers a reset via Supabase Auth.

3. NetSuite Integration Deep Dive

The portal communicates with NetSuite asynchronously via automated CSV generation. When a rep submits a request, a CSV is compiled and emailed to a processing address. This CSV is then manually

or automatically fed into specific NetSuite Suitelets.

Managing Suitelet Scripts

If business logic changes (e.g., adding a new field to an invoice), the SuiteScript must be updated.

1. In NetSuite, go to **Customization > Scripting > Scripts**.
2. Locate `\Zaks Portal Create Invoice from CSV`` (for DSD) or `\Zaks Portal Create Sales Order from CSV`` (for Standard Orders).
3. Edit the script record.
4. On the **Script File** dropdown, click the pencil icon to edit the file, and paste the updated JavaScript code.
5. Click **Save** on the file, then **Save** on the Script record. *(This purges NetSuite's execution cache).*

***** HARDCODED LOGIC & WAREHOUSE LOCKS *****

The portal contains strict logic guards to prevent NetSuite import failures:

1. **DSD Warehouse Lock:** Direct Invoices (DSD) are strictly locked to **Zaks - Edmonton Warehouse**.
 - The portal actively prevents submission if the cart quantity exceeds the live Edmonton inventory.
 - The Suitelet dynamically maps the string "Zaks - Edmonton Warehouse" to NetSuite Internal ID ``2``.
2. **Sales Order Lock:** Standard Sales Orders are locked to **Zaks - Main Warehouse YYC**.
3. **If warehouse structures change, you must update BOTH the Vercel Portal Code** (`\OrderBuilder.tsx``) and the NetSuite Suitelet (`\locationMap``).

4. Advanced Troubleshooting & Error Resolution

Portal-Side Errors

- **"Cannot submit DSD Invoice: Insufficient inventory..."**

***Cause:** The rep is attempting to sell more units than are physically tracked in the Edmonton warehouse.

***Fix:** The rep must reduce the quantity, or a NetSuite Admin must perform an Inventory Adjustment in NetSuite to reflect the actual physical stock in the driver's vehicle/Edmonton warehouse.

- **Activity Timeline Failures**

***Cause:** The Supabase `activities`` table has a strict database-level CHECK constraint on the `activity_type`` column to prevent data corruption.

***Fix:** Ensure the portal is only passing allowed enum strings (e.g., ``DRAFT_ORDER``, ``VISIT``, ``NOTE``). Custom strings like ``DSD_INVOICE`` will be rejected by the database engine.

NetSuite Import Failures

When processing a CSV through the Suitelet, NetSuite's native validation engines may throw errors:

- **"Please configure the inventory detail for this line"**

***Cause:** The target warehouse has "Use Bins" enforced. Direct Invoices cannot be created via CSV for bin-enforced locations.

Fix: Utilize the Edmonton warehouse bypass (which does not enforce bins), or manually create the invoice in the NetSuite UI.

- ****Pricing & Amount Errors****

Cause: The customer's assigned Price Level does not have a defined rate for a specific SKU.

Fix: Open the Item Record in NetSuite, navigate to the Pricing subtab, and ensure the customer's Price Level has a valid dollar amount.

- ****Missing SKUs Warning****

Cause: A product exists in the Portal but its SKU is mismatched or missing in NetSuite.

Fix: The Suitelet is programmed to gracefully skip the missing item, process the rest of the invoice, and append a massive `[WARNING: SKIPPED MISSING SKUs]` tag in the Memo. The admin must correct the SKU spelling in Supabase to match NetSuite.

- ****Closed Accounting Periods****

Cause: The financial month has been locked by the accounting team.

Fix: The invoice date must be manually adjusted to the current open period.